ClinicDesk AI: prompt documentation

Vishwa Singh · AI Growth & Automation Associate (WFH) · Task 1: AI Skills & Prompt Engineering · 2 November 2026

Project link: https://github.com/change3000/clinicdesk-ai · live app: https://change3000.github.io/clinicdesk-ai/

What I built

An AI triage assistant for an Australian GP clinic's patient inbox (Healthcare · Automation). For every message it (1) checks for emergencies and stops automation if one is found, (2) classifies the request into one of 9 types, (3) extracts details such as medicines, tests, doctor, dates and phone number, and (4) routes it to the right team with a drafted reply for staff to approve. The models are trained in Python (scikit-learn) and run in the browser, so no patient message leaves the device and reviewers need no login or API key.

AI tools and why

Tool	Used for	Why this tool
Claude (Anthropic), agent mode	Planning, writing every dataset, Python training and evaluation code, the JavaScript app, testing in a headless browser	It can write code, run it and read the results in one loop, so every claim below was measured, not guessed
Python + scikit-learn	TF-IDF + logistic regression intent model; word + character n-gram urgency model; evaluation	Small, explainable models that can be exported to JSON and run in a browser
Node.js + Playwright	Checking that the JavaScript engine gives the same answers as Python; screenshots at desktop/phone width, light/dark	Catches bugs a person would only find by clicking around

Part 1: my prompts to Claude (verbatim)

PROMPT 1 · WITH THE TASK 1 PDF ATTACHED

```
i got one reply , i have to submit within few hours
```

Claude asked two questions before building. My answers:

PROMPT 2 · MY SELECTIONS

```
Project: Clinic inbox AI
Video: I record, you prep
```

(Claude had offered three projects: a clinic inbox AI, a list of Australian GP practices, or an AI marketing campaign; and two video options.)

Context Claude used: the Task 1 brief, the job description (Naukri 221026000604: Python, healthcare automation, the MIRA project, web scraping of doctors' data) and my background (co-founder and product lead at Macbease, 40,000+ students; built an AI recommendation feed using semantic matching).

Part 2: the working prompts behind each step

Claude broke my brief into the step-by-step prompts below and ran them. Each one shows the result it produced and the revision it led to. The pattern throughout: build, measure on data the model has not seen, then change one thing.

Step 1 · Scoping

SCOPING PROMPT

```
Read the Task 1 brief and the job description. Pick one healthcare automation project
that: shows Python; runs without a server or API key so reviewers can open it; and is
honest about accuracy. Propose the request types an Australian GP clinic receives and a
pipeline that puts patient safety first.
```

Result: 9 request types (booking, change/cancel, repeat prescription, test results, referral, medical certificate, billing & Medicare, general enquiry, clinical question) plus an emergency check that runs before everything else.

Step 2 · First dataset (v1)

DATA PROMPT V1

```
Write 20 realistic patient messages per request type for an Australian GP clinic. Use
local terms: Medicare, bulk billing, script, chemist, WorkCover, telehealth, pathology.
Complete, polite sentences. No real people or patient data.
```

TEST-SET PROMPT

```
Write 10 more per request type in a different voice: SMS shorthand, typos, no punctuation,
slang ("arvo", "r u"). Do not reuse phrasing from the training messages.
```

Result: 180 training and 90 test messages. Model v1 scored **91.1%** on the test set. Almost every miss used shorthand the model had never seen ("bloods", "cert", "appt", "park" vs "parking").

What I changed next: Rewrote the data prompt to cover the way patients really text.

Step 3 · Messy data and a shorthand dictionary (v2)

DATA PROMPT V2 (REVISION)

```
Write 15 more training messages per request type in messy SMS style: abbreviations (appt,
pls, tmrw, rx, cert, referal), typos, very short messages, a symptom mentioned inside an
admin request. Avoid near-copies of the test messages.
```

LEAKAGE CHECK PROMPT

```
For every test message, find the most similar training message (cosine similarity). List
pairs above 0.55 and rewrite the training messages that are near-copies.
```

NORMALISATION PROMPT

```
Build one dictionary that maps SMS shorthand and common misspellings to standard words
(appt to appointment, script to prescription, referal to referral) and apply it
identically in Python and JavaScript.
```

Result: 8 near-copies rewritten so the test stayed fair. Model v2 scored **95.6%**.

Step 4 · Two experiments that did not work (v3, v4)

EXPERIMENT PROMPT

```
v3: add word pairs (bigrams) and balanced class weights. v4: add keyword boosts for
explicit requests ("cancel", "book", "certificate", "fee") on top of the model. Report
test accuracy and vocabulary size for each.
```

Result: v3 grew the vocabulary from 639 to 2,229 words with no gain (95.6%). v4 reached **98.9%**, which looked like the winner.

Step 5 · Checking on messages nobody had seen

GENERALISATION PROMPT

```
Write 27 brand-new messages (3 per request type) that are never used for training or
tuning. Score v1, v2 and v4 on them.
```

Result: v1 77.8%, v2 **88.9%**, v4 **74.1%**. The keyword boosts in v4 had been fitted to the test set and broke on new wording ("shifts" read as "move my appointment"). I shipped v2, the simplest model that generalised best.

What I changed next: Kept this habit for the rest of the project: every important number comes from data written after the change.

Step 6 · Knowing when to ask a person

CONFIDENCE PROMPT

```
Look at the model's top probability and how close the runner-up is, on all 117 test and
holdout messages. Find a rule that sends unsure messages to a person while letting most
routine ones through.
```

Result: Rule: hand to a person if the top probability is under 20% or the runner-up is within 85% of it. 17 of 117 messages go to a person (catching 5 of the 7 mistakes); the other 99 are routed automatically, with 97 correct (**98%**).

Step 7 · Emergency detection: the hardest part

SAFETY PROMPT V1

```
Write red-flag rules for emergencies a clinic receptionist must never miss: chest pain,
breathing difficulty, stroke (FAST) signs, severe allergic reaction, suicide or self-harm,
overdose, collapse, severe bleeding, seizure. Write 14 emergencies and 8 routine messages
that contain scary words ("chest x-ray results") to test them.
```

FRESH-TEST PROMPT (REPEATED AFTER EVERY REVISION)

```
Write new emergencies in words the rules have never seen, plus routine messages. Score the
current rules before changing anything.
```

SAFETY PROMPT V2 AND V3

```
Rewrite the rules around symptoms instead of exact phrases. Never put exclusions on
symptom rules; only ignore history or admin context for bare words such as "stroke" or
"seizure" ("stroke rehab referral"). Then widen: both word orders, "catch her breath",
"better off without me".
```

URGENCY MODEL PROMPT

```
Train a second, learned check on words and word fragments (character n-grams) using all
earlier emergencies and 345 routine messages. Pick the threshold, then test only on the
last unseen set.
```

Result: Rules scored perfectly on the messages they were written from, but on new wording caught only 4/10 (v1), 5/12 (v2) and 2/10 (v3). One bug was caught early: the pattern "end my" matched "send my script". Adding the learned model raised it to **8/10** emergencies caught, with 1 false alarm in 117 routine messages.

What I changed next: The 2 remaining misses (indirect suicide language; "nose pouring blood for 40 minutes") are routed to the nurse, so a person sees them. Before real use I would add an LLM safety prompt as a third check and test it the same way.

Step 8 · Replies and the app

REPLY PROMPT

```
Write one reply per request type for a fictional clinic (Harbourside Family Practice) in a warm, plain Australian tone. Never give clinical advice: clinical questions get a nurse callback plus 000 and Healthdirect (1800 022 222). Fill in details found (medicine, test, doctor, time). Use .example web addresses and the 02 5550 phone range reserved for fiction.
```

APP PROMPT

```
Build a single-file web app with four tabs: an inbox of 24 sample messages sorted by priority, a live 'try a message' box, batch triage with CSV in and out, and a 'how it works' page showing the real measured results. Run the Python-trained models in JavaScript and prove they give the same answers as Python.
```

TESTING PROMPT

```
Open the app in a headless browser at desktop and phone width, in light and dark mode. Report console errors and any sideways scrolling, and check the screenshots.
```

Result: JavaScript and Python agree exactly on accuracy, and the urgency scores match to within 0.00003. Testing caught two interface problems (long messages overlapping the time stamp; the 'key words' panel showing filler words like "my" and "what"), both fixed.

Results summary

Measure	Result
Request type, test set (90 messages)	95.6%
Request type, 27 messages written after the model was built	88.9%
Messages routed automatically that were correct	97 of 99 (98%); 17 unsure messages sent to a person
Emergencies caught in never-seen wording	8 of 10
Routine messages wrongly flagged as emergencies	1 of 117

All data is synthetic and written for this project, so real-world accuracy will be lower until the models are retrained on de-identified clinic messages.

What I learned about prompting

• **Describe the voice, not just the topic.** The first data prompt produced tidy sentences; real patients text "r my bloods back". Asking for a specific style (shorthand, typos, very short) fixed most errors.

• **Ask the AI to test itself on data it has not seen.** The most useful prompts were the ones that said "write new examples and score before changing anything". They exposed v4's overfitting and the weakness of keyword rules.

• **Ask for counter-examples.** Routine messages with scary words ("chest x-ray results", "stroke rehab referral") kept the emergency check from crying wolf.

• **Keep a person in the loop where it matters.** Every reply is a draft; unsure and clinical messages always go to a human.